

Exercise 4: Dimensionality Reduction and Clustering

Submission: Sunday, 3rd May, 23:59h

In Exercise 3, we observed that multivariate datasets become difficult to visualize as dimensionality increases. In this task, you will apply dimensionality reduction (Task 1) and clustering (Task 2) to the same multivariate dataset and visualize the clustering results (Task 4).

Task 1: Dimensionality Reduction

To apply dimensionality reduction, you can use all libraries included in `environment.yml` in this task.

a. **Download** the dataset in OLAT. The dataset contains the following variables:

- **Area:** area name
- **Year:** year
- **Item:** crop type
- **Yield_hg_ha:** crop yield (hectograms per hectare)
- **Rainfall_mm:** average annual rainfall (millimeters)
- **Temp:** average annual temperature (°C)
- **Pesticides_ton_ha:** total pesticide usage (tons per hectare)
- **Continent:** Continent of the area

Please note that there are NaN values in the dataset that should be dropped before you apply dimensionality reduction.

b. Perform PCA (Principal Component Analysis) on the four numerical attributes **Yield_hg_ha**, **Rainfall_mm**, **Temp**, and **Pesticides_ton_ha** of the **filtered** subset, i.e. allowing filtering by **Year** and **Item** as introduced in Task 5. Reduce the data to two principal component dimensions *only* on the filtered data.

1 point

Task 2: Clustering

In order to further explore the dataset, we try to cluster the areas based on the type of crop

1 point

of each year. The clustering method we use here is K-means. You can use all libraries included in environment.yml in this task.

- a. Perform K-means Clustering using the four variables **Yield_hg_ha**, **Rainfall_mm**, **Temp**, and **Pesticides_ton_ha** to group the filtered data points into **three** clusters.
- b. Perform K-means Clustering using the **first two PCA** components derived from the dimensionality reduction from Task 1, and again group the filtered data points into **three** clusters.

Task 3: Manual Implementation

In this task, you can choose to manually implement one of the algorithms used in Task 1 or Task 2.

1 point

- If you implement PCA, replace the sklearn PCA function with your own in Task 1 and repeat Task 2.
- If you implement K-means, replace the sklearn K-means function with your own in Task 2 and repeat the clustering process.

When implementing, you are **not allowed** to use the following functionalities of any library:

- Any **PCA or K-means** functionality
- **SVD** functionalities
- **Covariance** and **correlation** functionalities

You are **allowed** to use the following functionalities of **numpy** or **scikit-learn**:

- **Basic math** operations
- **Transpose** and **matrix multiplication**
- **Mean, norm, standard deviation, and variance**
- **Sort** and **argsort** functionalities
- **np.linalg.eig** or similar **eigen decomposition** functions on matrices
- **Optional**: Implement both for a bonus point (including their use in Task 4 and Task 5).

1 bonus point

Task 4: Visualization

Use a scatter plot to display the clustering results. You can use all libraries included in environment.yml in this task.

- Use the **first two PCA components** as x- and y-axis (**Note**: make sure to use the same scale for both axes so that distances are proportional and comparable.)

1 point

- Use color to encode **clusters**
- Use symbol to encode **Continent** (**Note:** if the symbols are not enough, define your own symbol list.)

You should produce:

- 2 plots using Scikit-learn implementations
- 1 plot using your manual implementations
- **Optional:** If you implement both manual PCA and manual K-means, include the corresponding plot.

Compare the results and briefly discuss any differences in a .txt or .pdf file.

Task 5: Filtering

1 point

Add interactive filters:

- **Year:** slider
- **Item:** dropdown menu

Filtering must be applied **before PCA and clustering**, so that all computations are based on the selected subset. You can use all libraries included in environment.yml in this task.

Hand-in

Please hand in your solution in OLAT as a **.zip** file until **Sunday, 3rd May, 23:59h**. The .zip file should include the following:

- Your **python code** with the executable final application in a single file (.py) with a reasonable amount of comments explaining your implementation.
Note: You are **not** allowed to use **other libraries** despite the ones in environment.yml.
- Include a few **screenshots** demonstrating your running application.
- Required .txt or .pdf file. In case there are additional remarks, include them as a .txt or .pdf file too.
- Please hand-in your own unique solutions. We do not accept copied solutions or solutions created by or with ChatGPT!